

# CS 342: Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

# Shanti Mantra

## Lyrics in Sanskrit

## Meaning in English

ॐ सह नाववतु।

Om, May we all be protected

सह नौ भुनक्तु।

May we all be nourished

सह वीर्यं करवावहै।

May we work together with great energy

तेजस्यि नावधीतमस्तु

May our intellect be sharpened (may our study be effective)

मा विद्विषावहै।

Let there be no Animosity amongst us

ॐ शान्तिः शान्तिः शान्तिः ॥

Om, peace (in me), peace (in nature), peace (in divine forces)



# Upsampling

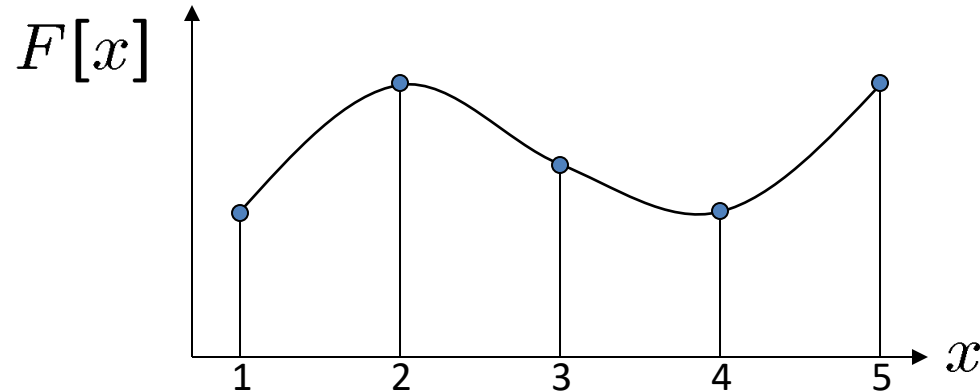
- This image is too small for this screen: 
- How can we make it 10 times as big?
- Simplest approach:
  - repeat each row
  - and column 10 times
- (“Nearest neighbor interpolation”)



# Interpolation Techniques

- 1D linear interpolation (elementary algebra)
- 2D = 2 sequential 1D (divide-and-conquer)

# Image interpolation



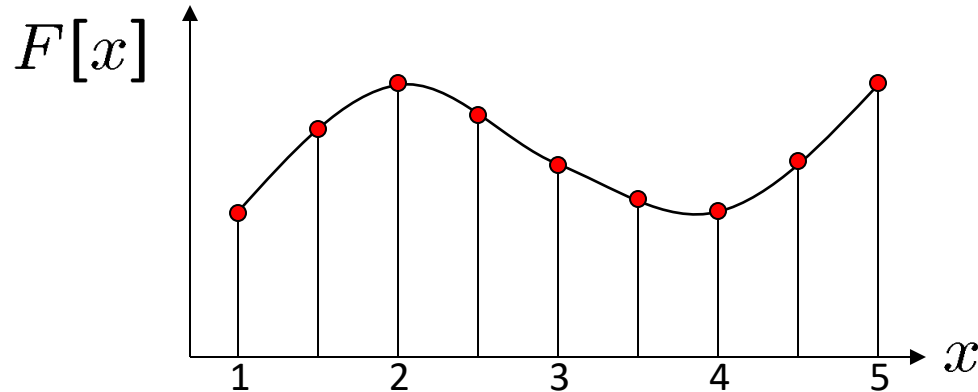
$d = 1$  in this example

Recall how a digital image is formed

$$F[x, y] = \text{quantize}\{f(xd, yd)\}$$

- It is a discrete point-sampling of a continuous function
- If we could somehow reconstruct the original function, any new image could be generated, at any resolution and scale

# Image interpolation



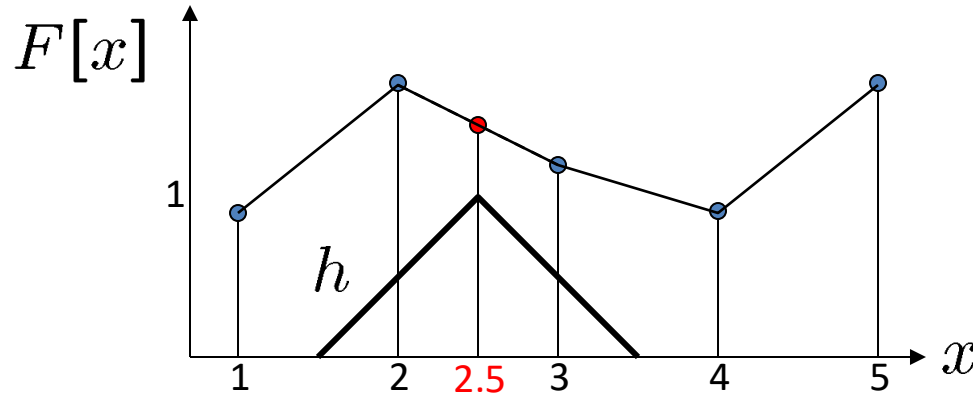
$d = 1$  in this example

Recall how a digital image is formed

$$F[x, y] = \text{quantize}\{f(xd, yd)\}$$

- It is a discrete point-sampling of a continuous function
- If we could somehow reconstruct the original function, any new image could be generated, at any resolution and scale

# Image interpolation



$d = 1$  in this example

- What if we don't know  $f$  ?

- Guess an approximation:  $\tilde{f}$
- Can be done in a principled way: filtering
- Convert  $F$  to a continuous function:

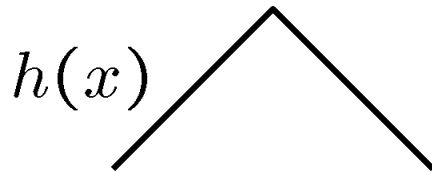
$$f_F(x) = F\left(\frac{x}{d}\right) \text{ when } \frac{x}{d} \text{ is an integer, } 0 \text{ otherwise}$$

- Reconstruct by convolution with a *reconstruction filter*,  $h$

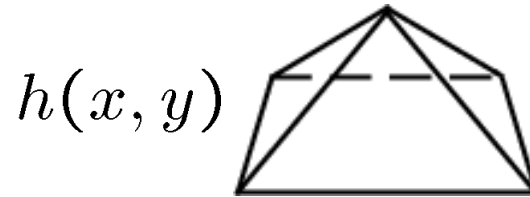
$$\tilde{f} = h * f_F$$

# Resampling filters

- What does the 2D version of this hat function look like?



performs  
linear interpolation



(tent function) performs  
**bilinear interpolation**

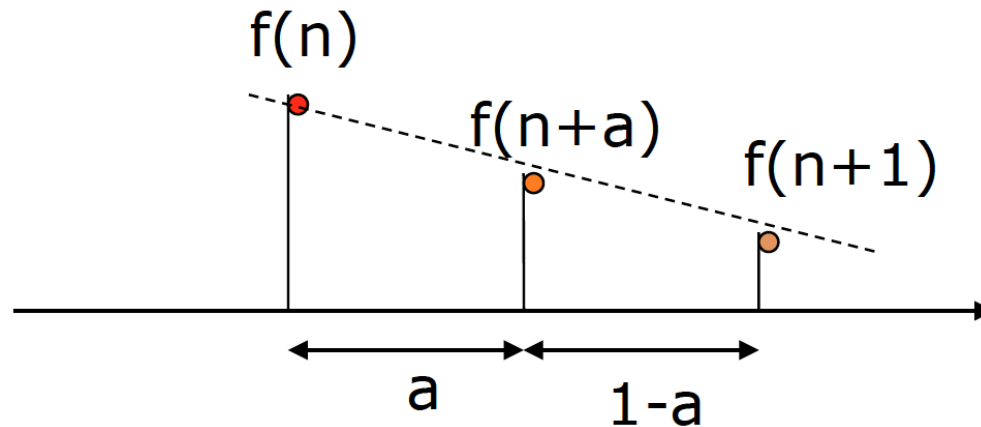
## Better filters give better resampled images

- Bicubic is common choice
  - fit 3<sup>rd</sup> degree polynomial surface to pixels in neighborhood
  - can also be implemented by a convolution



# Linear Interpolation (Idea)

Heuristic: the closer to a pixel, the higher weight is assigned  
Principle: line fitting to polynomial fitting (analytical formula)

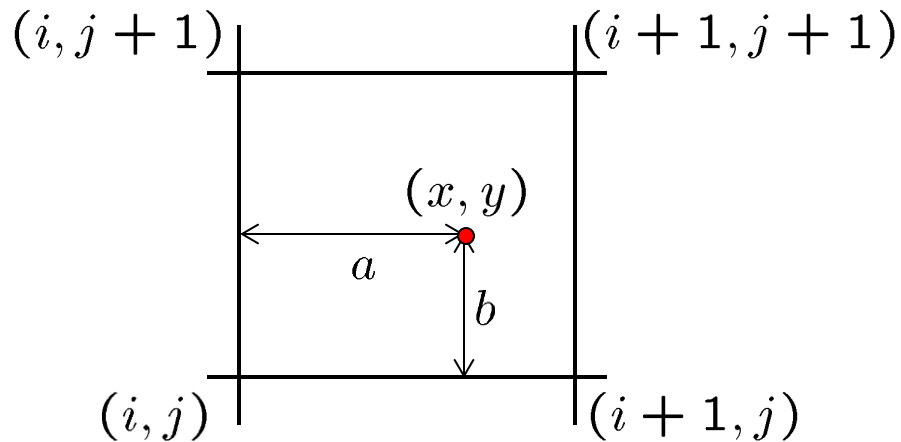


$$f(n+a) = (1-a) \times f(n) + a \times f(n+1), \quad 0 < a < 1$$

Note: when  $a=0.5$ , we simply have the average of two

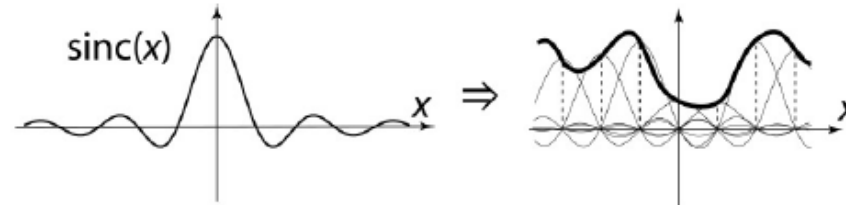
# Bilinear interpolation

- A simple method for resampling images

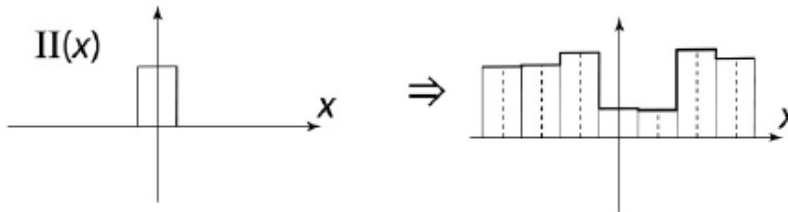


$$\begin{aligned} f(x, y) = & (1 - a)(1 - b) f[i, j] \\ & + a(1 - b) f[i + 1, j] \\ & + ab f[i + 1, j + 1] \\ & + (1 - a)b f[i, j + 1] \end{aligned}$$

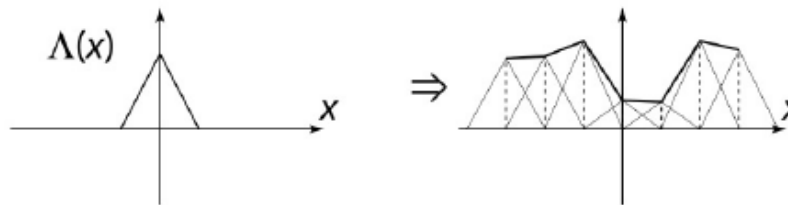
# Image interpolation



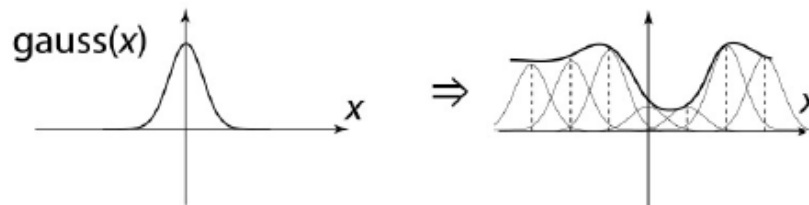
“Ideal” reconstruction



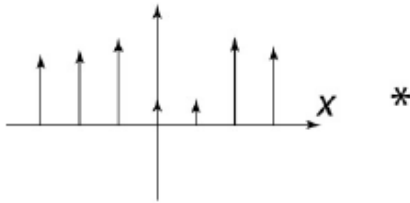
Nearest-neighbor interpolation



Linear interpolation



Gaussian reconstruction



# Image interpolation

Original image:  x 10



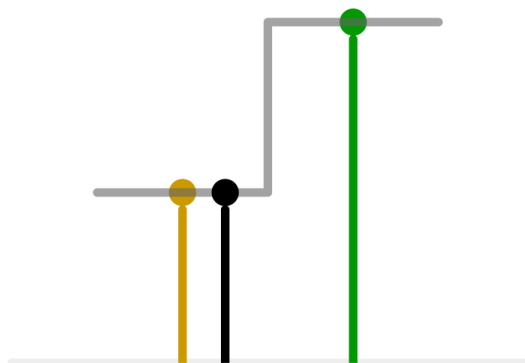
Nearest-neighbor interpolation



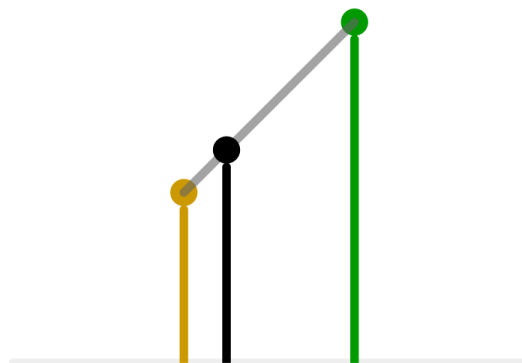
Bilinear interpolation



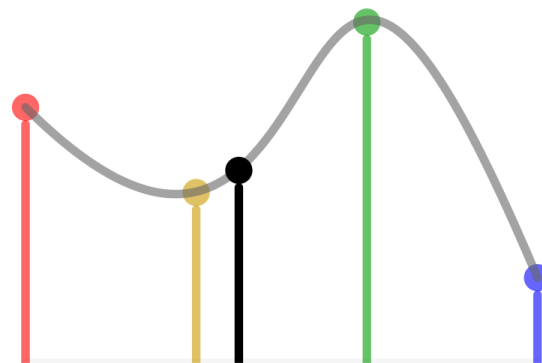
Bicubic interpolation



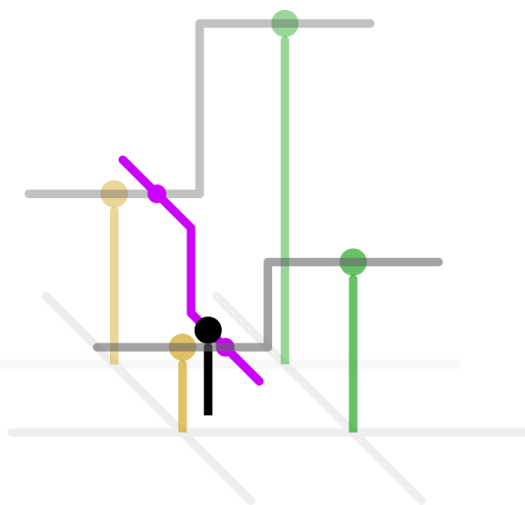
1D nearest-neighbour



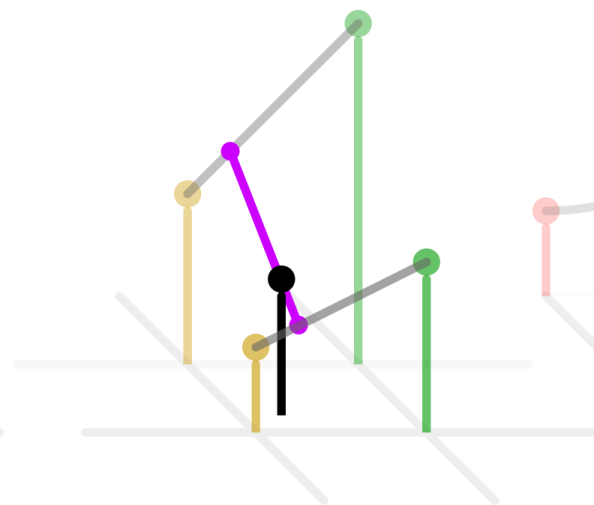
Linear



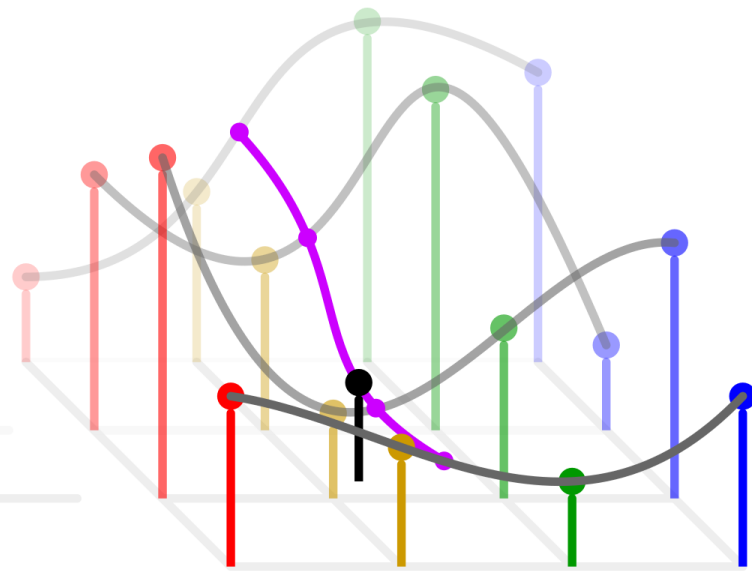
Cubic



2D nearest-neighbour



Bilinear



Bicubic

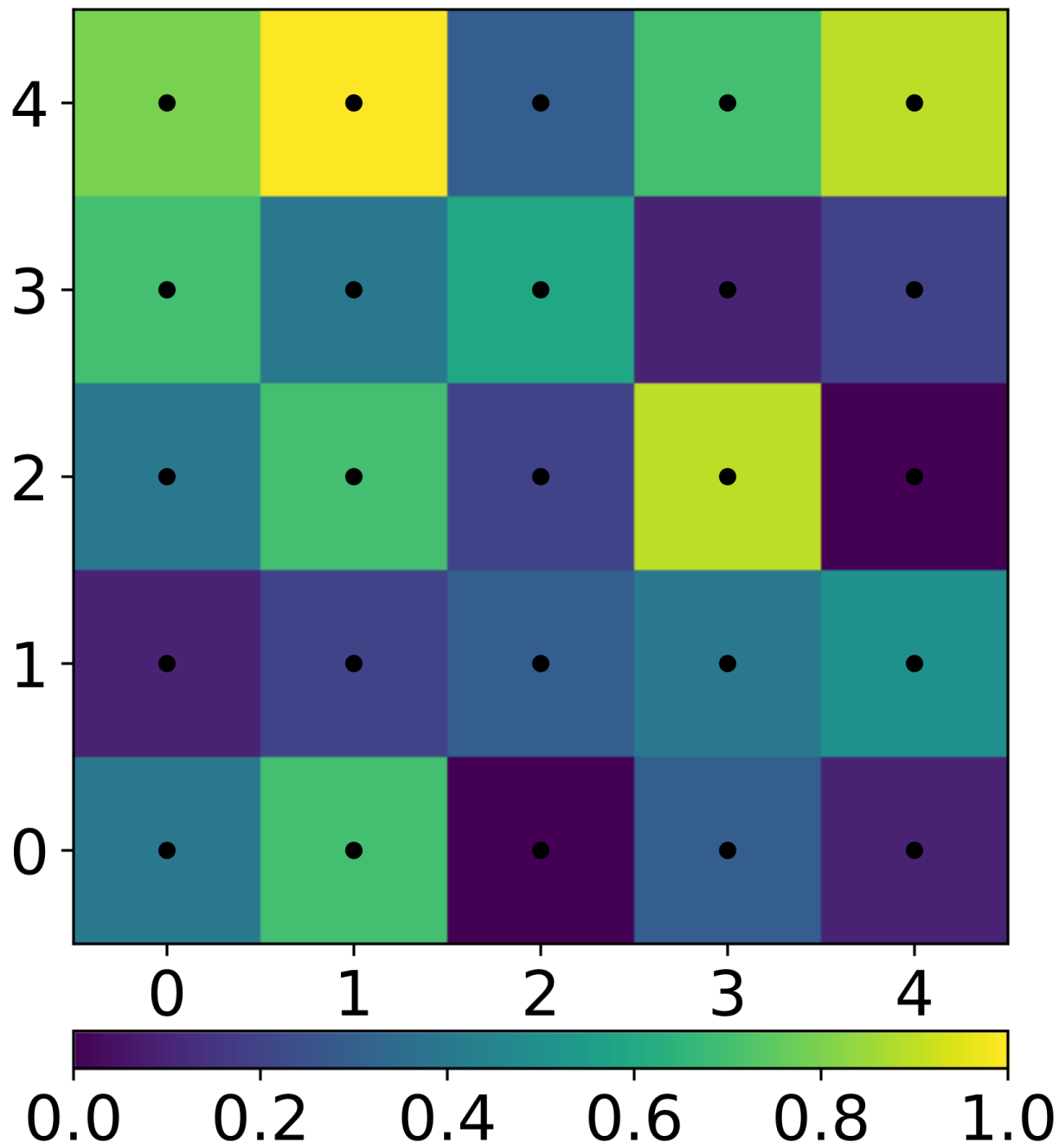
# Nearest Neighbor Interpolation Kernel

- 1D

$$h(x) = \begin{cases} 1 & 0 \leq |x| < 0.5 \\ 0 & 0.5 \leq |x| \end{cases}$$



# nearest

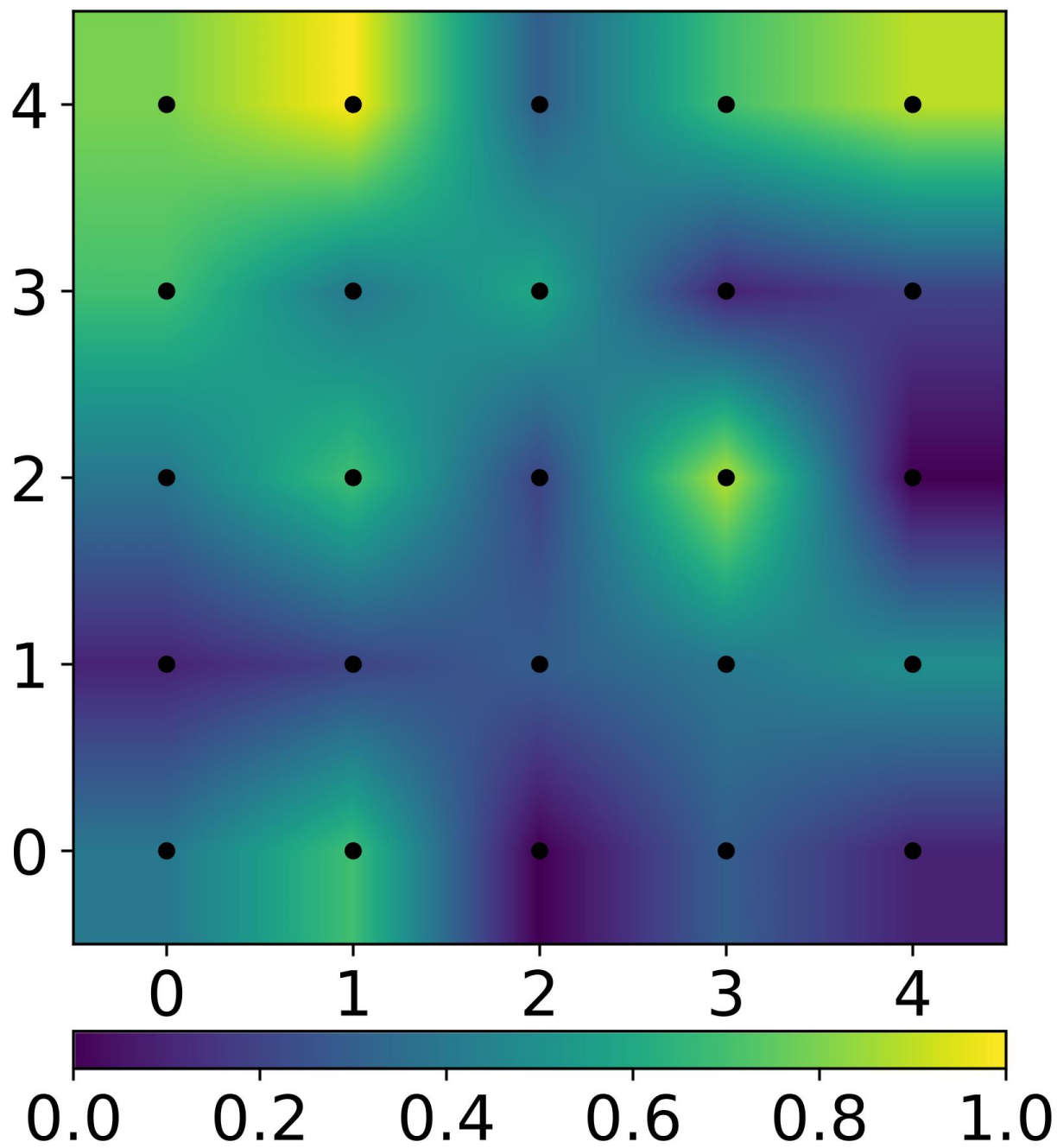


# Linear Interpolation Kernel

- 1D

$$h(x) = \begin{cases} 1 - |x| & 0 \leq |x| < 1 \\ 0 & 1 \leq |x| \end{cases}$$

# bilinear

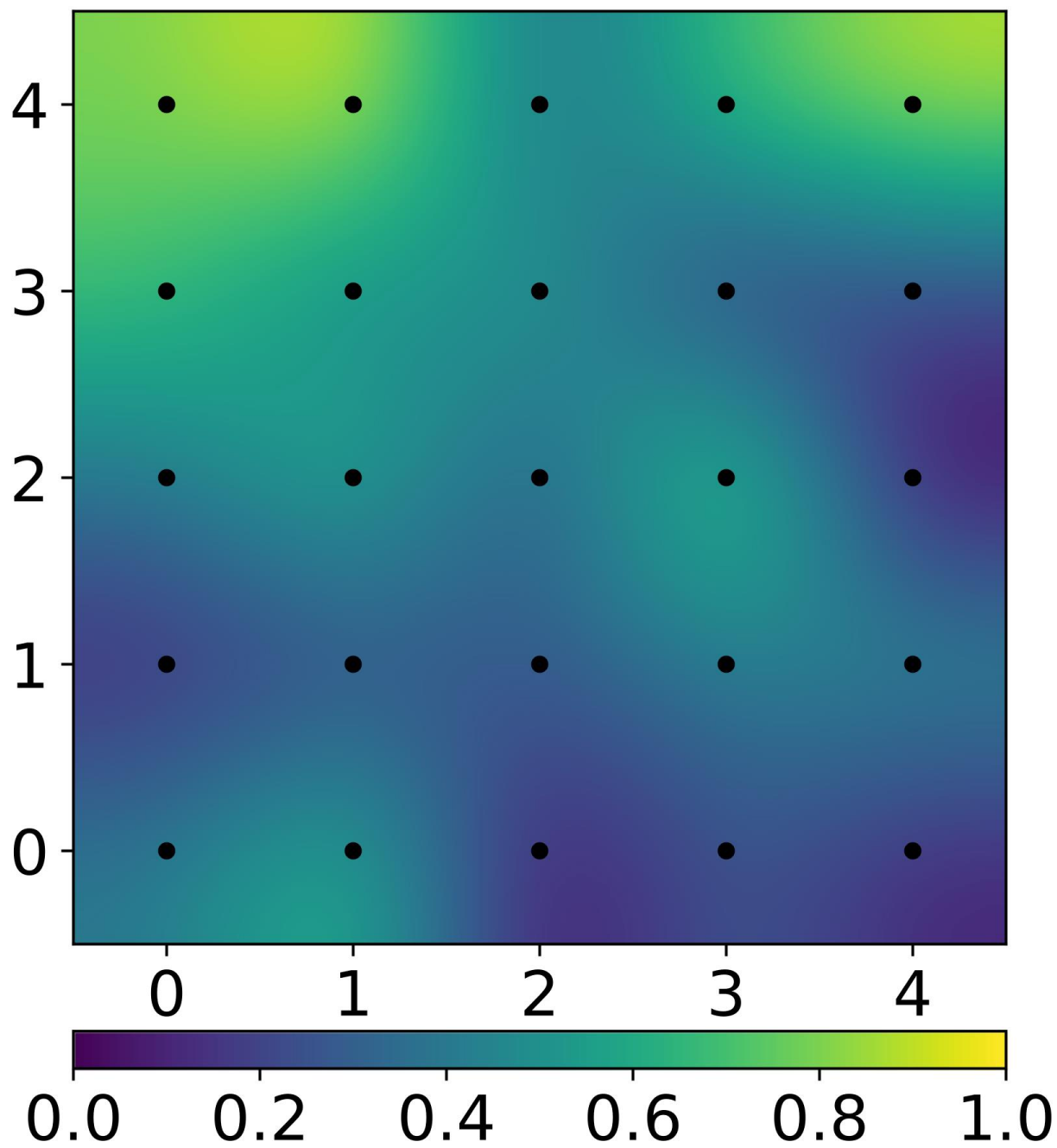


# Cubic Interpolation Kernel

- 1D

$$h(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1 & 0 < |x| < 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a & 1 < |x| < 2 \\ 0 & 2 < |x| \end{cases}$$

# bicubic



# Image interpolation

Original image:  x 10



Nearest-neighbor interpolation

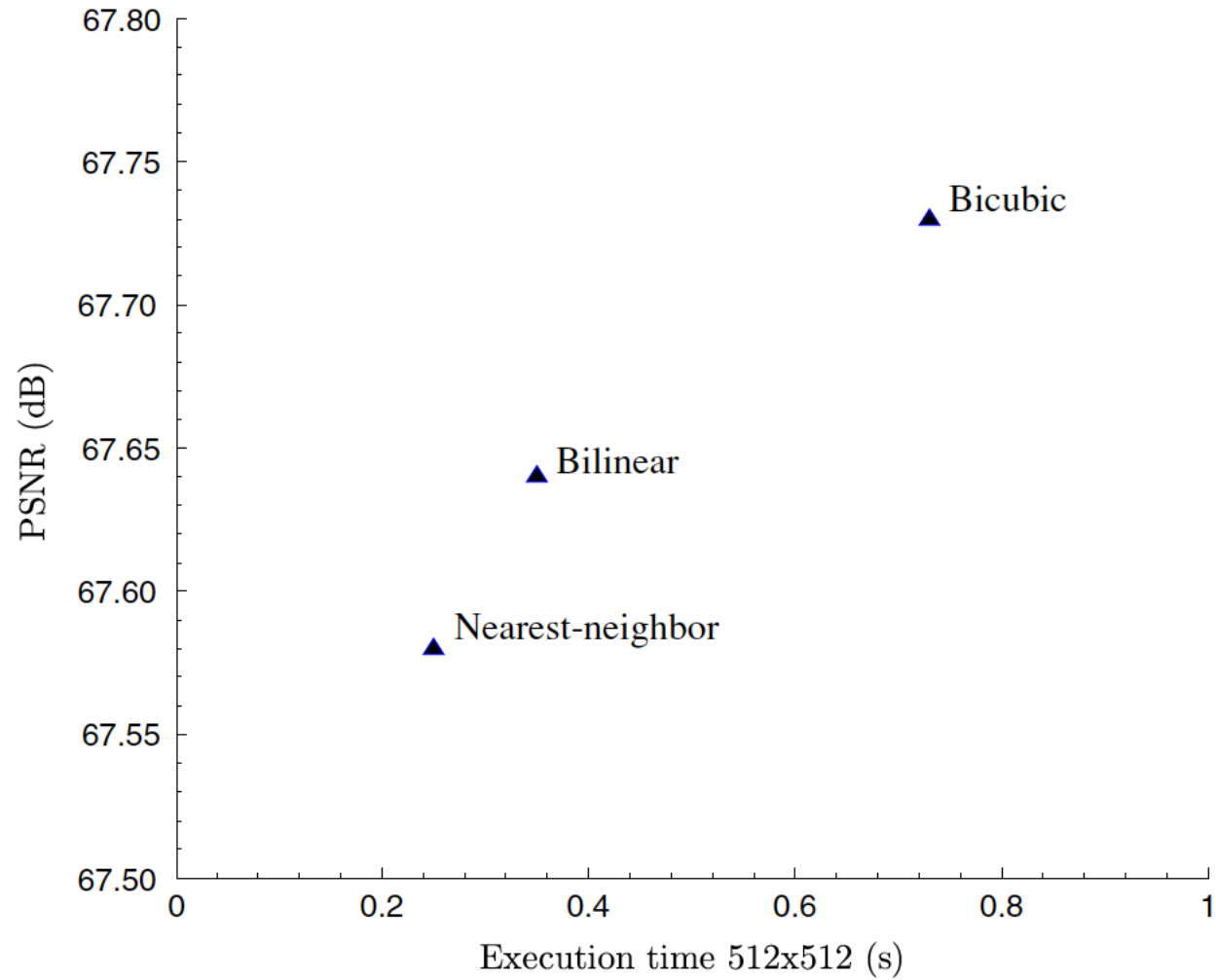


Bilinear interpolation



Bicubic interpolation

# Performance



# OpenCV options for Interpolation

- `INTER_NEAREST`: Nearest neighbor Interpolation
- `INTER_LINEAR`: Bilinear Interpolation
- `INTER_CUBIC`: Bicubic Interpolation
- `INTER_LANCZOS4`:
  - Lanczos kernel to compute a weighted average over a 8x8 region
  - sinc-based function to maintain edge sharpness. So, slow!
- `INTER_AREA`:
  - Averages all pixels in the corresponding area
  - [Excellent for extreme downsampling;]



# How Lanczos4 Works

- **Uses a windowed sinc function** to compute the new pixel value.
- Takes into account **8 neighboring pixels** (4 on each side, hence "Lanczos4").
- Provides **sharp, high-quality results**, especially for **downscaling** images.
- Computationally **more expensive** than bilinear or bicubic interpolation.

Mathematically, the **Lanczos kernel** is:

$$L(x) = \begin{cases} \frac{\sin(\pi x) \sin(\pi x/a)}{(\pi x)^2}, & \text{if } |x| < a \\ 0, & \text{otherwise} \end{cases}$$

where  $a = 4$  in **Lanczos4**.

# Comparison of Interpolation Methods

| Method         | Uses                 | Computation | Best For                 | Upsampling Quality    | Downsampling Quality      |
|----------------|----------------------|-------------|--------------------------|-----------------------|---------------------------|
| INTER_NEAREST  | Fast, rough          | ✅ Fastest   | Pixel art, binary images | ❌ Poor (jagged edges) | ❌ Poor (loss of details)  |
| INTER_LINEAR   | Smooth, balanced     | ✅ Fast      | General resizing         | 👍 Decent (blurry)     | 👍 Good (some detail loss) |
| INTER_CUBIC    | Sharp, detailed      | 🔹 Slower    | High-quality upscaling   | ✅ Sharp & smooth      | ✅ Good                    |
| INTER_LANCZOS4 | High-end quality     | 🔴 Slowest   | Best quality upscaling   | ✅✅ Very sharp         | ✅ Good                    |
| INTER_AREA     | Best for downscaling | 🔹 Medium    | Downsampling             | ❌ Not suitable        | ✅✅ Best                   |